Implement a Deep Neural Network (DNN) to solve the XOR problem

To implement a Deep Neural Network using TensorFlow/Keras and train it to learn the XOR logical operation.

Algorithm
1. Import NumPy and TensorFlow libraries.
2. Define the four possible binary input combinations.
3. Define the corresponding XOR output values.
4. Convert the input and output data into NumPy arrays.
5. Create a Sequential neural network model.
6. Add an input layer with two neurons and a hidden layer with four neurons using ReLU activation.
7. Add an output layer with one neuron using sigmoid activation.
8. Compile the model using the Adam optimizer and binary cross-entropy loss.
9. Train the model using the XOR dataset for multiple epochs.
10. Predict the output for all four input combinations.

11. Convert the predicted probabilities into binary values using a threshold of 0.5.
12. Display the actual and predicted outputs.

Program

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
tf.keras.utils.set_random_seed(7)
# XOR dataset
X = np.array([
[0, 0],
[0, 1],
[1, 0],
[1, 1]
], dtype=np.float32)
y = np.array([
[0],
[1],
[1],
[0]
], dtype=np.float32)
# Create DNN model
model = Sequential([
Dense(4, input_shape=(2,), activation='relu'),
Dense(1, activation='sigmoid')
])
# Compile model
model.compile(
optimizer=tf.keras.optimizers.Adam(learning_rate=0.03),
loss='binary_crossentropy',
metrics=['accuracy']
)
# Train model
model.fit(X, y, epochs=1500, verbose=0)
# Prediction
predictions = model.predict(X, verbose=0)
print("XOR Predictions")
for i in range(len(X)):

result = int(predictions[i][0] >= 0.5)
print(
X[i].astype(int),
"Expected:", int(y[i][0]),
"Predicted:", result
)
```

Output of Program

```
XOR Predictions
[0 0] Expected: 0 Predicted: 0
[0 1] Expected: 1 Predicted: 1
[1 0] Expected: 1 Predicted: 1
```

```
[1 1] Expected: 0 Predicted: 0
```

Result

Thus, a Deep Neural Network was successfully implemented and trained to learn the XOR logical operation. The model correctly predicted the output for all four input combinations.